

Задание на лабораторные работы по курсу «Технологии сетевого программирования»

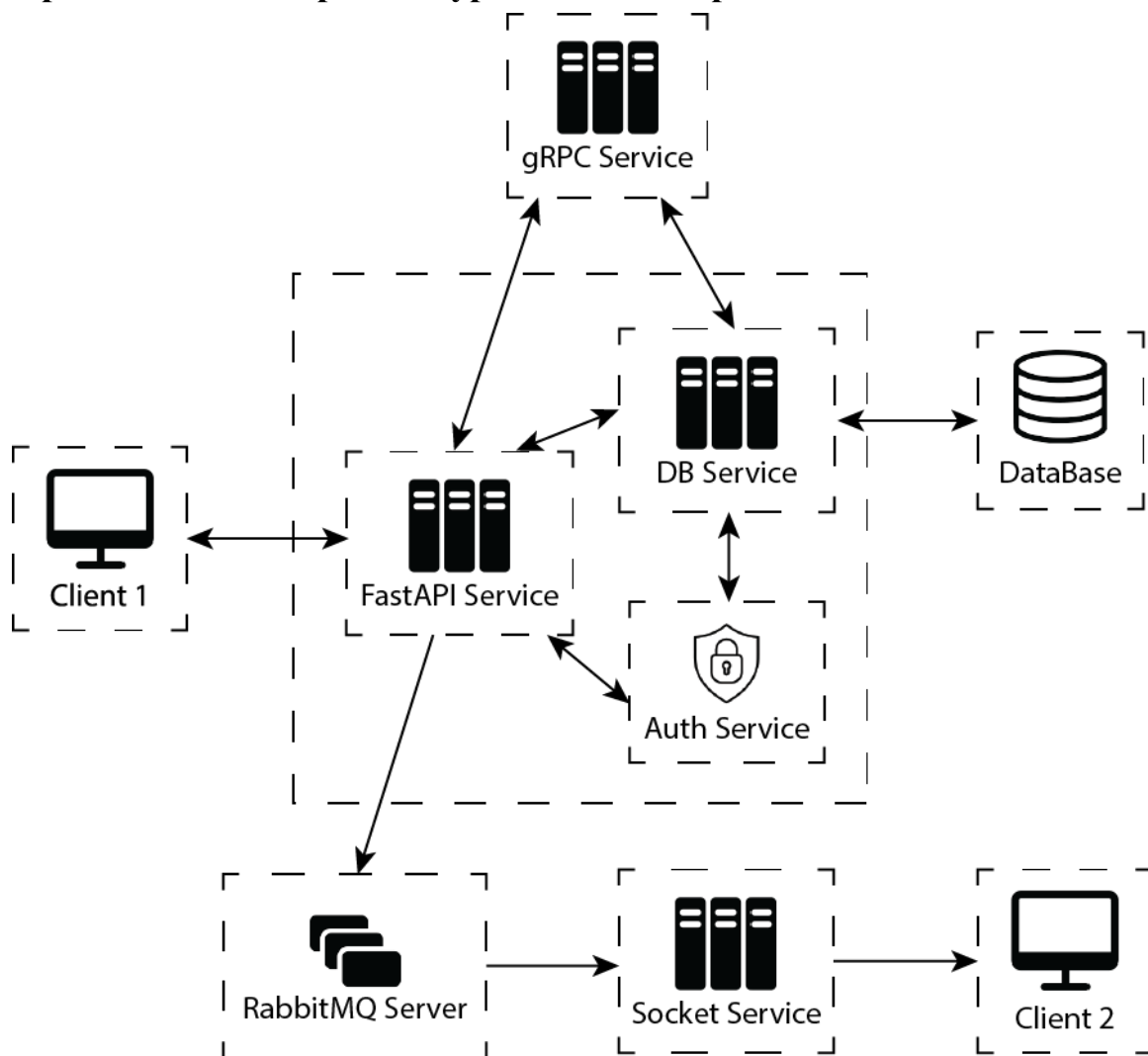
Цель курса — изучить основные модели взаимодействия компонентов распределённой системы на практике путём разработки собственного многокомпонентного приложения.

В ходе выполнения лабораторных работ требуется разработать распределённое приложение выбранной предметной области, состоящее из нескольких взаимодействующих компонентов, с использованием различных сетевых протоколов и способов обмена данными. Предметная область и структура базы данных – на усмотрение разработчиков. Проект выполняется в командах по 2 человека.

Итоговый проект задействует следующие сетевые технологии:

1. SQLAlchemy – библиотека для работы с реляционной БД
2. Pydantic – библиотека для валидации и сериализации данных
3. FastAPI — веб-фреймворк для взаимодействия клиентского приложения с сервером средствами REST API
4. gRPC — синхронное взаимодействие сервисов внутри распределённой системы
5. RabbitMQ — асинхронный брокер сообщений для обмена событиями между компонентами
6. WebSocket — доставка событий клиентам в режиме реального времени
7. JWT-аутентификация — защита сетевых интерфейсов
8. JSON и Protocol Buffers — различные форматы сериализации данных
9. Docker — контейнеризация и развёртывание распределённой системы

Предполагаемая архитектура итогового проекта:



Интернет-магазин. Клиент может просматривать каталог товаров и историю своих заказов, а также создавать новые заказы. Администратор может просматривать историю заказов любого клиента. Рекомендательный сервис на основе gRPC предлагает клиенту товары в соответствии с его историей заказов. Сервис оповещений уведомляет курьера о создании нового заказа посредством асинхронного брокера сообщений и сокета.

Примеры предметных областей:

- Социальная сеть. Пользователь может просматривать и лайкать посты других пользователей, на которых он подписан, а также создавать новые посты. Администратор может банить пользователей.
- Новостной портал. Пользователь может просматривать новостные статьи, фильтруя их по тегам. Администратор может загружать новые статьи и снабжать их тегами.

- Класный журнал. Учитель может просматривать расписание занятий, отмечать отсутствовавших учеников и ставить им оценки. Ученик может просматривать данные о своей успеваемости.
- Стриминговый сервис. Пользователь может просматривать каталог (фильмов, сериалов, музыкальных композиций) и покупать понравившиеся наименование, добавляя его к своему списку доступных наименований. Автор/исполнитель может загружать новые произведения и просматривать статистику скачиваний своих работ.
- Поликлиника. Пациент может записаться к врачу и просматривать историю своих посещений всех врачей. Врач может просмотреть историю визитов конкретного пациента и внести данные о результатах визита.

Порядок сдачи и выставление оценок

Отчёт по каждой из лабораторный работ осуществляется лично, в соответствии с требованием формы отчета. При этом оцениваются отдельно код и знание теории.

Оценка за код может быть снижена за некорректность и неработоспособность программы, несоответствие формальным требованиям, отступления от сроков контрольных точек, а также за низкое качество самого кода (запутанный код, избыливающий скопированными фрагментами и «костылями»).

Оценка за теорию может быть снижена по результатам ответов на дополнительные вопросы преподавателя о работе программы и использованных в ней технологиях.

Результатом выполнения лабораторных работ является программное обеспечение информационной системы и отчет о выполнении работы. Зачет выставляется после демонстрации работающей информационной системы, предоставления ссылки на репозиторий с исходным кодом программы и отчета о выполнении работы.

Лабораторная работа №0

Тема

Выбор и описание предметной области. Проектирование функционала приложения.

Цель

Сформулировать техническое задание к своему проекту.

Задание

- Выбрать предметную область. Сформулировать бизнес-цель приложения.
- Описать функционал приложения. Описать сценарий использования (какие конкретно действия должен совершить пользователь для доступа к функционалу).
- Описать архитектуру приложения: из каких основных компонент будет состоять приложение, каковы связи между компонентами, какие технологии будут использоваться для реализации компонент (с обоснованием выбора). Последнее – если стек технологий отличается от предложенного в лекционном курсе.
- Для выбранной предметной области спроектировать логическую и физическую схему базы данных, позволяющую реализовать выбранный функционал приложения.

Формальные требования

Проект должен обеспечивать:

- 1) возможность чтения, записи и редактирования данных в БД;
- 2) хотя бы две роли пользователей;
- 3) хотя бы три отдельные сущности в БД, и хотя бы одну связь типа многие-ко-многим.

Форма отчётности

Документ, в котором приводятся ответы на поставленные в задании вопросы. Рекомендуется использовать иллюстрации и диаграммы в стандартных нотациях.

Лабораторная работа №1

Тема

Проектирование базы данных и реализация слоя доступа к данным.

Цель

Изучить принципы хранения данных и работу ORM.

Используемые технологии

- PostgreSQL
- SQLAlchemy

Задание

Реализовать слой приложения, отвечающий за работу с базой данных.

- Создать и настроить БД PostgreSQL.
- В соответствии со разработанной в Л/р№0 схемой БД реализовать модели данных средствами SQLAlchemy. Реализовать CRUD-операции для основных сущностей.
- Реализовать функционал работы с БД в соответствии с выбранным функционалом с использованием реализованных моделей.
- Осуществить подключение к созданной БД, заполнить её тестовыми данными.
- Написать тестовый скрипт, демонстрирующий работу с БД во всех требуемых функционалом сценариях.

Дополнительное задание

Настроить миграцию БД с помощью Alembic. Продемонстрировать работу с инструментом миграции.

Форма отчётности

Продемонстрировать работающее приложение, демонстрирующее функционал работы с БД. Результаты запросов можно выводить в консоль (пользовательский интерфейс писать не обязательно).

Лабораторная работа №2

Тема

Разработка REST API.

Цель

Изучить HTTP как основной протокол взаимодействия клиентских приложений с сервером.

Используемые технологии

- Nginx
- Gunicorn
- uWSGI
- FastAPI
- Pydantic
- JWT

Задание

Реализовать REST API для работы с БД средствами HTTP. Разработанный API должен обеспечивать:

- Регистрацию и аутентификацию пользователей.
- CRUD операции над объектами в БД через REST-интерфейс.
- Прочие запросы к БД в соответствии с функционалом проекта.

В ответ на запрос сервер отправляет данные в формате json (если применимо).

Примеры запросов:

POST	/register
POST	/login
GET	/products
POST	/products
GET	/orders
POST	/orders

Реализовать API Service, реализующий обработку пользовательских запросов использованием фреймворка FastAPI.

При реализации обработчиков HTTP-запросов рекомендуется использовать механизм внедрения зависимостей (Dependency Injection), предоставляемый FastAPI, для получения экземпляров сервисов и репозиториев.

Использовать разделение моделей для HTTP-запросов (DTO) и для слоя работы с данными (ORM).

Дополнительное задание №1

Настроить базовый прокси-сервер Nginx, работающий в качестве обратного прокси (Reverse Proxy). Прокси-сервер должен принимать запрос пользователя, возвращать статические данные либо перенаправлять запрос на бэкенд сервер для дальнейшей обработки.

Дополнительное задание №2

Реализовать аутентификацию и авторизацию пользователей через JWT токены.

Форма отчётности

Продемонстрировать работающее приложение, позволяющее пользователю взаимодействовать с БД исключительно посредством HTTP-запросов. Запросы отправляются через Postman, либо через cURL.

Лабораторная работа №3

Тема

Сервисное взаимодействие с использованием gRPC.

Цель

Изучить RPC-подход и контрактное описание интерфейсов.

Используемые технологии

- gRPC
- Protocol Buffers

Задание

Создать отдельный gRPC-сервис, предоставляющий проекту дополнительный функционал в соответствии с бизнес-логикой приложения. FastAPI-сервер вызывает методы этого микросервиса в процессе обработки запроса средствами gRPC.

Примеры:

- рекомендации товаров;
- поиск похожих постов/материалов;
- применение купонов и пересчет стоимости заказа;
- анализ статистики;
- подбор исполнителей.

Разработанный сервис должен:

- Иметь собственный gRPC-интерфейс.
- Описывать интерфейс через .proto файл.
- Принимать запросы от API Service.
- Возвращать результат вычислений через gRPC.

Форма отчётности

Продемонстрировать работу приложения. Должен быть виден процесс обработки пользовательского запроса и то, как в этом процессе участвует gRPC-сервис. Данные о работе достаточно выводить в консоль.

API Service и gRPC-сервис – отдельные сервисы, запущенные в отдельных процессах.

Лабораторная работа №4

Тема

Асинхронный обмен сообщениями и уведомления в реальном времени.

Цель

Изучить событийно-ориентированную архитектуру и обмен сообщениями через брокер.

Используемые технологии

- RabbitMQ
- WebSocket

Задание

Дополнить проект функционалом уведомлений. Для этого разработать сервис уведомлений (Notification Service) и клиентское приложение, получающее уведомление.

При возникновении значимых событий API Service должен публиковать сообщения в брокер.

Разработать Notification Service, который должен:

- Подписываться на очередь сообщений.
- Получать события из RabbitMQ.
- Формировать уведомления для пользователей.
- Предоставлять WebSocket-интерфейс.
- Передавать уведомления подключённым клиентам.

Клиентское приложение, получающее уведомление, должно:

- Подключаться к WebSocket-серверу.
- Получать уведомления в режиме реального времени.
- Отображать полученные сообщения.

Форма отчётности

Продемонстрировать работу приложения. Должен быть виден процесс обработки пользовательского запроса и то, как в этом процессе участвует сервис уведомлений. Данные о работе достаточно выводить в консоль.

API Service, Notification Service и уведомляемый клиент – отдельные процессы.

Лабораторная работа №5

Тема

Контейнеризация и развёртывание распределённой системы.

Цель

Изучить принципы развёртывания многокомпонентных приложений.

Используемые технологии

- Docker
- Docker Compose

Задание

Развернуть все компоненты системы в отдельных контейнерах. Контейнеры, которые должны получиться (минимум):

- PostgreSQL
- API Service
- gRPC-Service
- RabbitMQ
- Notification Service (+ WebSocket)

Настроить:

- сетевое взаимодействие контейнеров;
- переменные окружения;
- постоянное хранение данных PostgreSQL;
- автоматический запуск всей системы через docker-compose.

Форма отчётности

Продемонстрировать работу приложения.

Вся система должна запускаться одной командой `docker compose up` и обеспечивать корректное взаимодействие всех компонентов.

Лабораторная работа №6

Дополнительная

Тема

Разработка пользовательского веб-интерфейса средствами серверного рендеринга.

Цель

Изучить принципы построения пользовательского интерфейса на стороне сервера (Server-Side Rendering, SSR) и интеграцию веб-интерфейса с существующей архитектурой приложения.

Используемые технологии

— Jinja

Задание

Дополнить разработанную распределенную систему, реализовав HTML-интерфейс с использованием шаблонизатора Jinja2. Разработанный веб-интерфейс:

- позволяет пользователю выполнять основные операции предметной области через браузер (отправлять запросы и видеть результаты)
- разрабатывается на основе уже существующего REST API
- использует существующую бизнес-логику приложения.

Для формирования HTML использовать шаблонизатор Jinja2. Шаблоны должны содержать только логику представления данных. В шаблонах допускаются вывод данных, циклы, условные конструкции, наследование шаблонов. Размещение бизнес-логики и обращений к базе данных в шаблонах не допускается.

Реализовать HTML-представления и маршрутизацию для основных сценариев предметной области (главная страница, просмотр списка объектов, просмотр карточки объекта, создание нового объекта, редактирование объекта и т.д.)

Новый функционал будет частью API Service, новых сервисов и контейнеров создавать не нужно. Создание отдельного фронтенд-сервиса, дублирование бизнес-логики не допускается. Все действия пользователя должны обрабатываться существующим сервисным слоем приложения.

Форма отчётности

Продемонстрировать работу приложения.

Оформление отчёта

Отчет должен быть оформлен в соответствии с СТО Самарского университета и здравым смыслом.

В отчете должны присутствовать следующие разделы:

- Введение – задачи создания информационной системы и краткое описание предметной области.
- Сценарии использования приложения – раздел, описывающий связь функций приложения с бизнес-целями и сценариями использования. Бизнес-цель должна отвечать на вопрос «что?» и быть измеримой. Для достижения бизнес-цели ваше приложение реализует функциональность – отвечает на вопрос «как?». Описание сценария использования – конкретные действия пользователя, которые он совершает, чтобы получить доступ к функциям приложения.
- Структура базы данных – описание логической и физической структуры базы данных с использованием диаграмм (нотация – на выбор).
- Архитектура приложения – краткое описание применяемых технологий, обоснование их использования для решения поставленной задачи, виды элементов системы, способы их взаимодействия. Архитектурные решения описываются с использованием диаграмм (предпочтительны диаграммы UML, а именно диаграммы компонентов, классов и т.д.).
- Приложения – примеры исходного кода модулей, конфигурационные файлы (для однотипных модулей можно приводить код только одного из них). Ссылка на github-репозиторий с проектом